A.E.D. Individual 2

Guillermo Román
guillermo.roman@upm.es

Julio García
juliomanuel.garcia@upm.es

Lars-Åke Fredlund
lfredlund@fi.upm.es

Tonghong Li
tonghong@fi.upm.es

Manuel Carro
mcarro@fi.upm.es

Francisco de Borja Lozano
francisco.lozano@upm.es

Normas.

- Fechas de entrega y penalización:

Hasta el Martes 6 de octubre, 16:00 horas	0
Hasta el Miércoles 7 de octubre, 16:00 horas	20 %
Hasta el Jueves 8 de octubre, 16:00 horas	40 %

  Después la puntuación máxima será 0
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender
- Se comprobará plagio y el uso no permitido de IA's, y se actuará sobre los casos detectados

Entrega

- Todos los ejercicios de laboratorio se deben entregar a través de

`http://deliverit.fi.upm.es`

- El/los fichero(s) que hay que subir es/son `Patterns.java`.
- La clase debe estar en el paquete `aed.individual2`
- La documentación de la API de `aedlib.jar` está disponible en

`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Ejecuciones

A muchos nos da miedo el comportamiento de los IA. Por eso hemos decidido desarrollar una funcionalidad para advertir cuando la ejecución de un modelo de IA no cumple con una determinada especificación.

Una *ejecución* de un IA es un `PositionList<Set<String>>`:

- La lista representa una secuencia de "momentos en el tiempo"
- En cada momento, la lista guarda las acciones que la IA ejecutó en ese momento, representado estas acciones como un *conjunto* – `Set` – de `String` (notad que una acción es un `String`)
- Por ejemplo,

```
[ { "close_door" }, { "move" }, { "move", "turn"}, { "stop" } ]
```

- En el primer momento de tiempo ocurrió la acción `close_door`
- En el tercer momento de tiempo ocurrieron las acciones (sin orden) `move` y `turn`

¿Qué es un Set<E>?

- La interfaz "Set<E>" modela un conjunto:
  - No tiene elementos repetidos.
  - No tiene orden entre los elementos.
- Para usarlo debéis importar la interfaz Set<E>:

```
import es.upm.aedlib.set.Set;
```

```
public interface Set<E> extends Iterable<E> {
  // Contiene el conjunto e?
  public boolean contains(E e);
  ...
}
```

Especificación

Para verificar que la IA no ha hecho algo prohibido tenéis que implementar 4 métodos para comprobar determinadas propiedades de una ejecución:

- `boolean exists(String s, PositionList<Set<String>> l)`

  devuelve `true` si existe un nodo dentro `l` cuyo conjunto contiene el `String s`.

- `boolean universal(String s, PositionList<Set<String>> l)`

  devuelve `true` si todos los conjuntos dentro de `l` contienen la acción `String s`.

Especificación

Para verificar que la IA no ha hecho algo prohibido tenéis que implementar 4 métodos para comprobar propiedades sobre una ejecución:

- `boolean precededBy(String s1, String s2, PositionList<Set<String>> l)`

  devuelve `true` si para cualquier nodo dentro `l` que contiene `s1`, hay un nodo anterior, es decir, "hacia la izquierda" (`prev`) en `l` que contiene `s2`

- `boolean followedBy(String s1, String s2, PositionList<Set<String>> l)`

  devuelve `true` si para cualquier nodo dentro de `l` que contiene `s1`, hay un nodo posterior, es decir, hacia la derecha" (`next`) en `l` que contiene `s2`

Método `deleteStates`

Además hay que implementar un método para modificar una `PositionList`:

- ```
  void deleteStates(String s, PositionList<Set<String>> l)
  ```

  debería borrar de `l` todos los momentos en el tiempo (nodos) cuyas acciones (un conjunto) contienen `s`
  ```

Ejemplos

```
// Se asume que l = [ { "close_door" }, { "move" }, { "turn", "move" },
//                    { "open_door" }, { "stop" } ]

Patterns.exists("move", l);       // => true
Patterns.exists("moving", l);     // => false
Patterns.universal("move", l);    // => false

Patterns.universal("move", []);   // => true
Patterns.exists("move", []);      // => false

Patterns.precededBy("move", "close_door", l);    // => true
Patterns.precededBy("open_door", "stop", l);     // => false
Patterns.followedBy("move", "stop", l);          // => true
Patterns.followedBy("turn", "move", l);          // => false
Patterns.precededBy("move", "turn", l);          // => false

Patterns.deleteStates("move", l);
// => l = [ { "close_door" }, { "open_door" }, { "stop" } ]
```

Notas importantes

- El valor de los parámetros `l` nunca serán `null` y nunca contendrá elementos `null`; similarmente los conjuntos nunca contiene elementos `null`
- **No se debe modificar la estructura de datos recibida como parámetro**, excepto cuando explícitamente lo permite en el enunciado (p.e., para `deleteStates`).
- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar.
- Debe ejecutar `TesterInd2` correctamente sin mensajes de error.
- Nota: una ejecución sin mensajes de error **no** significa que el método sea correcto (es decir, que funcione bien para cualquier entrada posible).